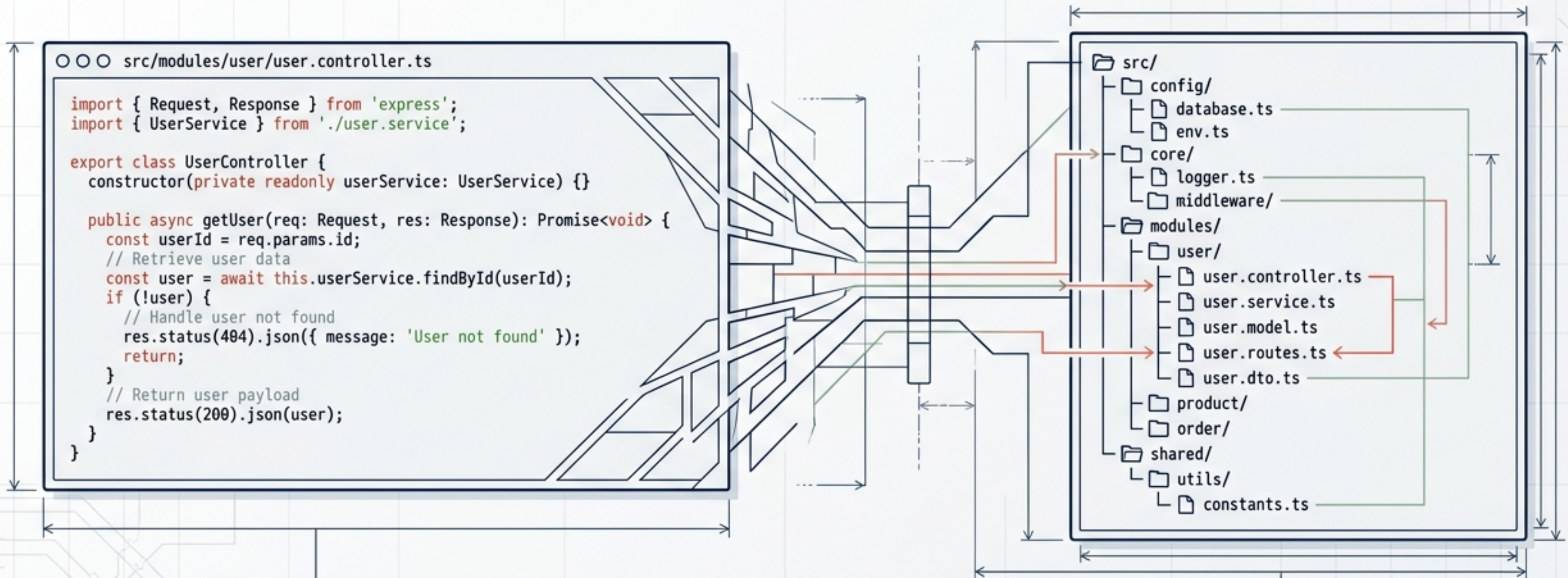
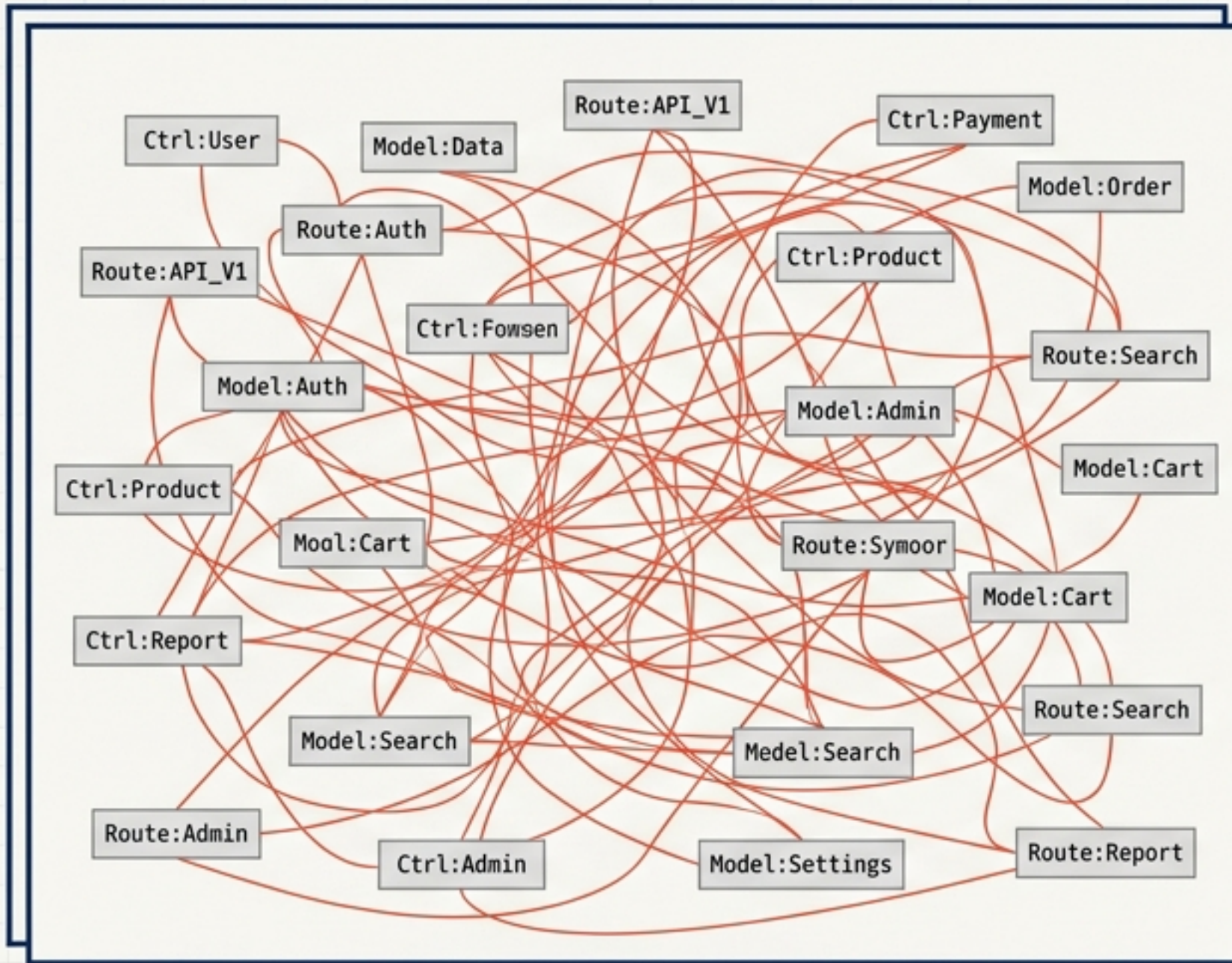


Architecting Scalable APIs with Express.js and TypeScript



Building feature-driven, microservice-ready architectures using modern ES6 modules.

The Monolith Bottleneck

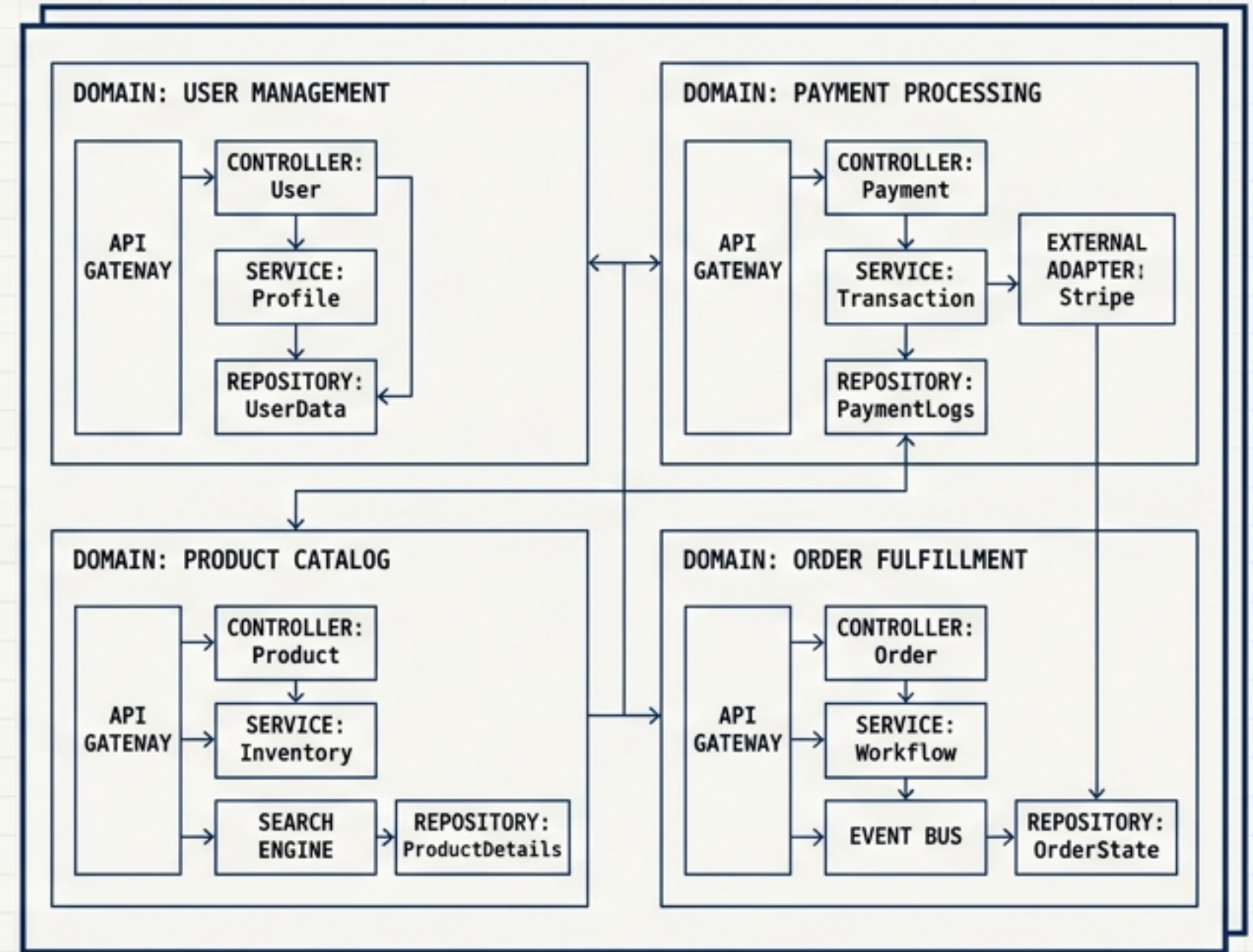


Tangled Dependencies

Horizontal slicing creates invisible, unpredictable couplings.

Scaling Dead-End

Impossible to seamlessly extract heavily coupled features into standalone microservices.



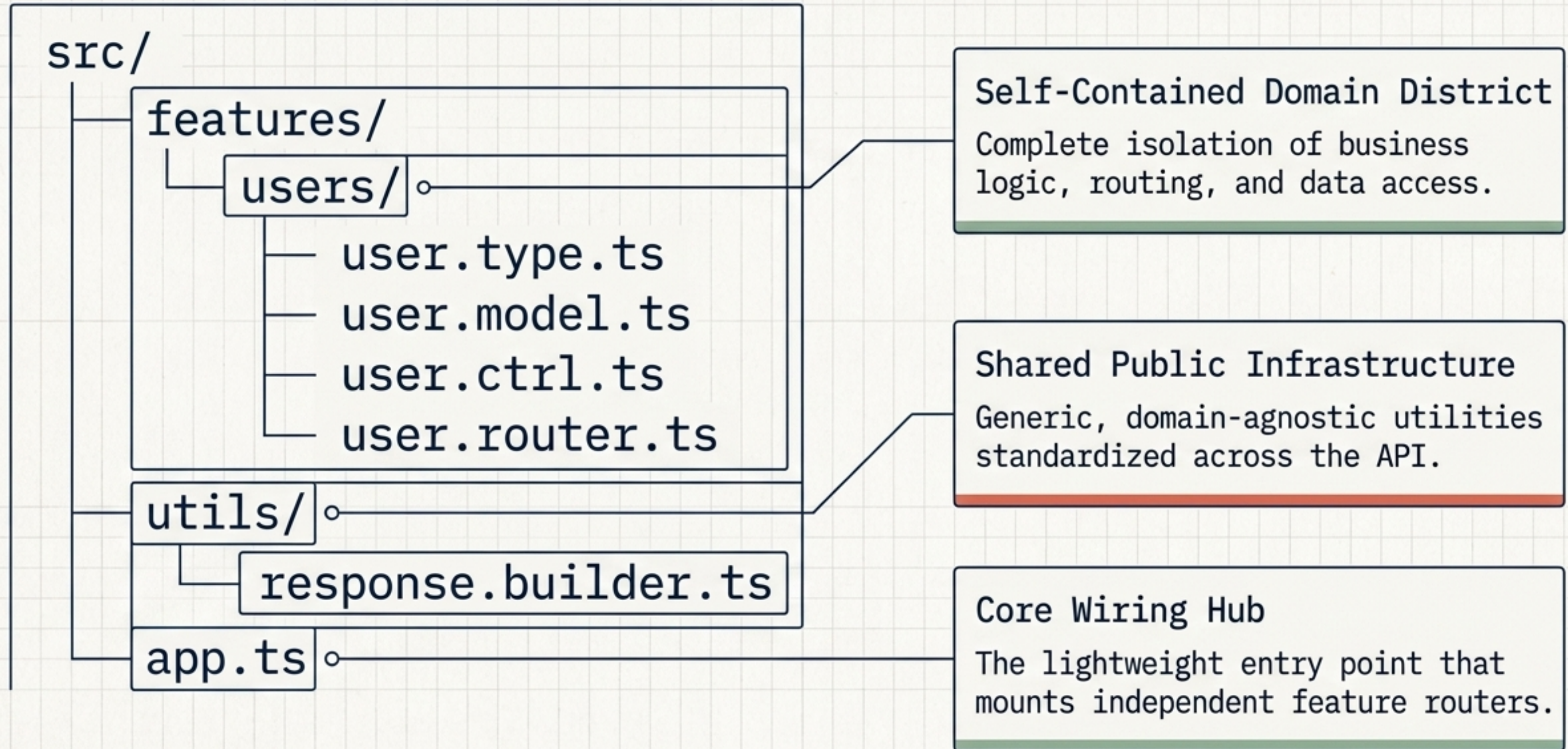
Refactoring Friction

Modifying one domain risks breaking another without strict isolation.

Horizontal MVC vs. Vertical Feature Slicing

	Traditional Horizontal MVC	Simulated Microservices (Vertical Slicing)
Code Navigation	✗ Jump between 4 root folders to trace one feature	✓ Everything for a feature is contained in one directory
Refactoring Ease	✗ High risk of cross-domain breakage	✓ Safe, isolated, and predictable domain boundaries
Scalability	✗ Locked into a monolithic codebase	✓ Ready for immediate extraction to separate Node.js services
Data Contracts	✗ Loose, global types prone to bleed	✓ Strict, feature-scoped TypeScript interfaces

The Microservice-Simulated Directory Tree

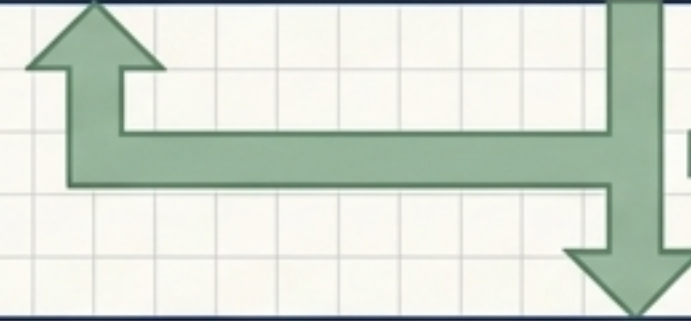


The Foundation: ES6 Modules & TypeScript

package.json

```
{  
  'name': 'express-feature-api',  
  'type': 'module',  
  'dependencies': { 'express': '^4.18.2' }  
}
```

```
// Exposing internal logic safely  
export const fetchData = async () => { ... }
```

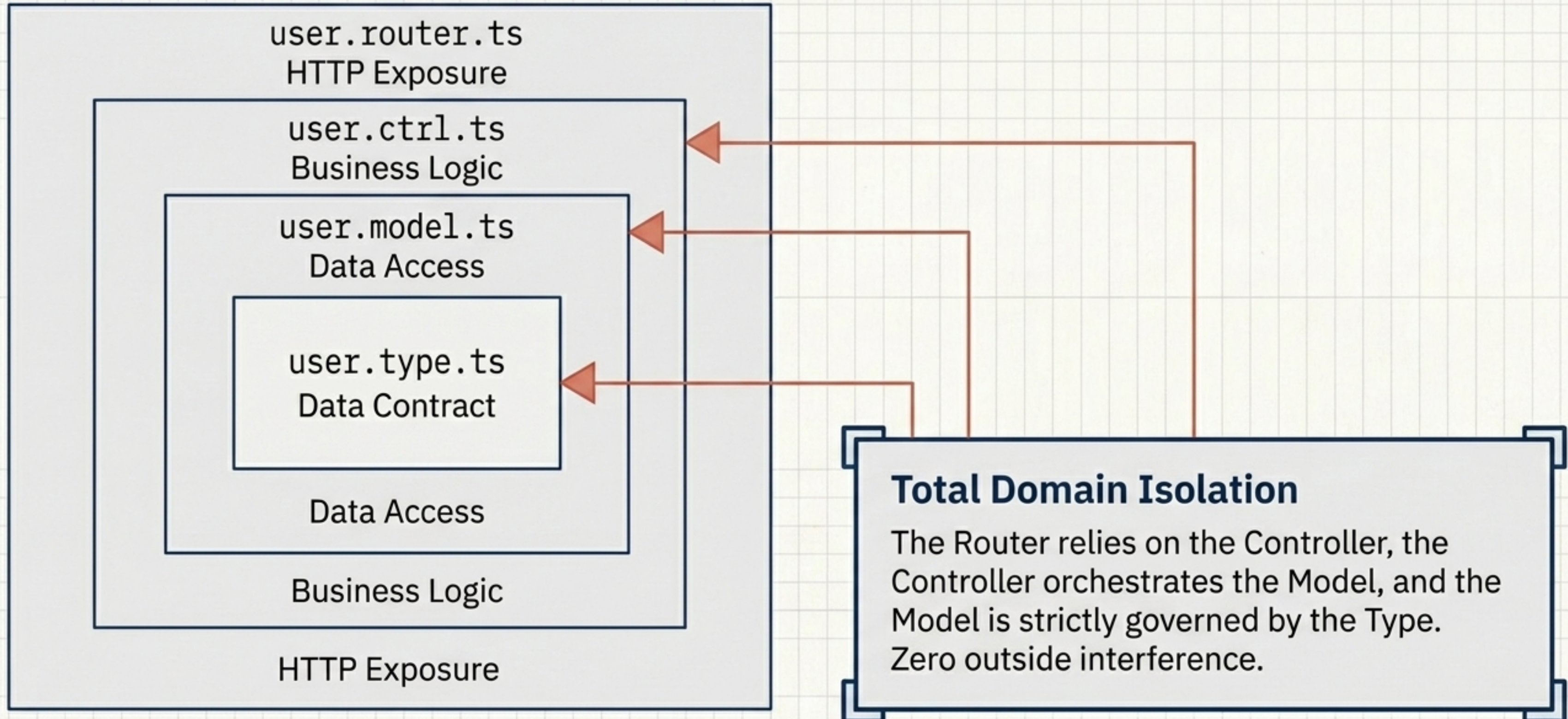


ES6 Bridge (import/export)

```
// Consuming logic in another file  
import { fetchData } from './user.model.js';
```

Setting 'type': 'module' enables modern ES6 architecture, ditching legacy require() for strict, statically analyzable dependency chains that support tree-shaking.

Anatomy of a Feature Module



Layer 1: Defining the Contract

user.type.ts

```
// user.type.ts
export interface User {
  id: string;
  email: string;
  name: string;
  role: 'admin' | 'customer';
  createdAt: Date;
}

export interface CreateUserDTO {
  email: string;
  name: string;
}
```



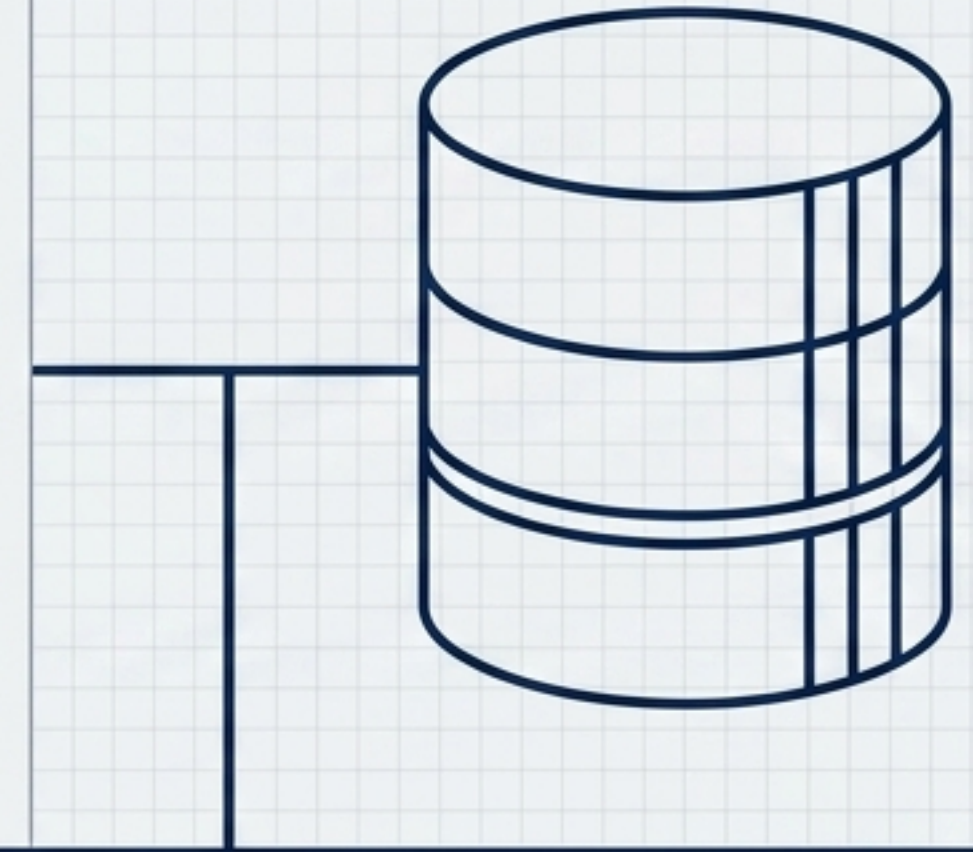
TypeScript interfaces define the exact shape of the data. By exporting these types via ES6, we guarantee an unbreakable data contract across the entire micro-domain.

Layer 2: Data Access

```
// user.model.ts
import { User, CreateUserDTO } from './user.type.js';

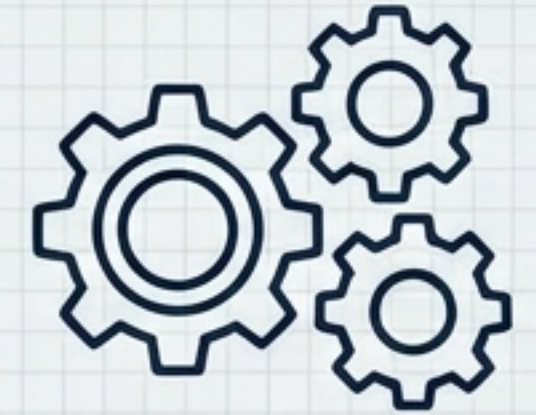
const usersDb: User[] = [];

export const insertUser = async (data: CreateUserDTO): Promise<User> => {
  const newUser: User = {
    id: crypto.randomUUID(),
    ...data,
    role: 'customer',
    createdAt: new Date()
  };
  usersDb.push(newUser);
  return newUser;
};
```



The Model handles all direct data manipulation. It imports the strict 'User' type, ensuring nothing invalid ever enters or exits the database layer.

Layer 3: Business Logic



```
// user.ctrl.ts
import { Request, Response } from 'express';
import { insertUser } from '../user.model.js';
import { buildSuccess } from '../../utils/response.builder.js';

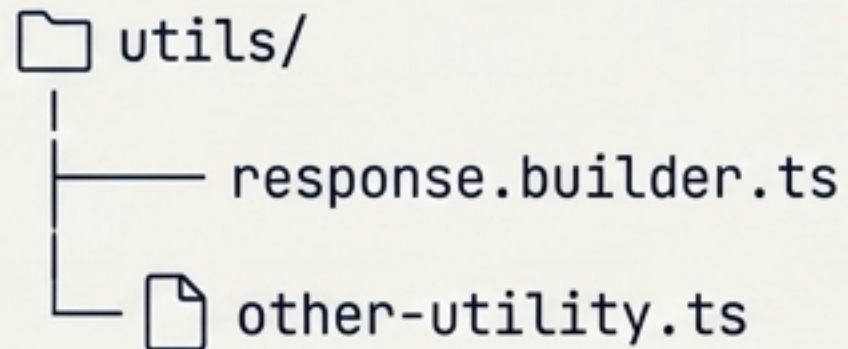
export const createUser = async (req: Request, res: Response) => {
  try {
    const userData = req.body;
    const newUser = await insertUser(userData);

    return res.status(201).json(buildSuccess(newUser));
  } catch (error) {
    return res.status(500).json({ error: 'Creation failed' });
  }
};
```

Controllers orchestrate the process. They extract the request data, pass it to the Model, and format the final outcome using shared generic utilities, totally decoupled from routing.

Shared Infrastructure: Generic Utilities

Shared Utilities



```
// response.builder.ts
export interface ApiResponse<T> {
  success: boolean;
  data?: T;
  error?: string;
  timestamp: string;
}

export const buildSuccess = <T>(data: T): ApiResponse<T> => {
  return {
    success: true,
    data,
    timestamp: new Date().toISOString()
  };
};
```

ES6 exported generic functions provide standard infrastructure. The '<T>' generic ensures that the payload remains strictly typed, even in a shared utility file used across domains.

Layer 4: Exposing the API

```
// user.router.ts
import { Router } from 'express';
import { createUser } from './user.ctrl.js';

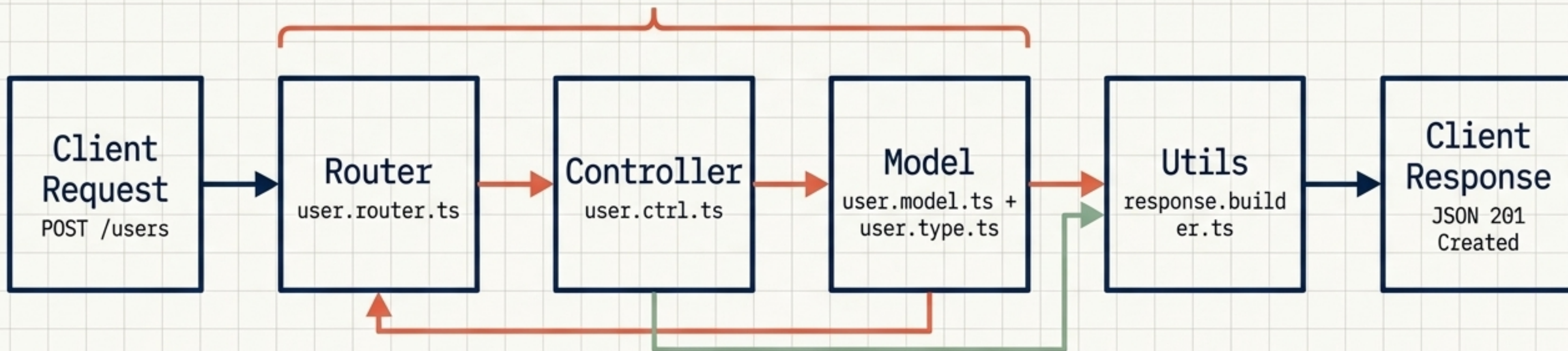
export const userRouter = Router();

// Map HTTP methods directly to Controller logic
userRouter.post('/', createUser); POST
// userRouter.get('/:id', getUserById); GET
```

The Router acts purely as a traffic cop. By utilizing ES6 import, it neatly maps incoming HTTP requests to isolated controller functions, keeping the file lean and readable.

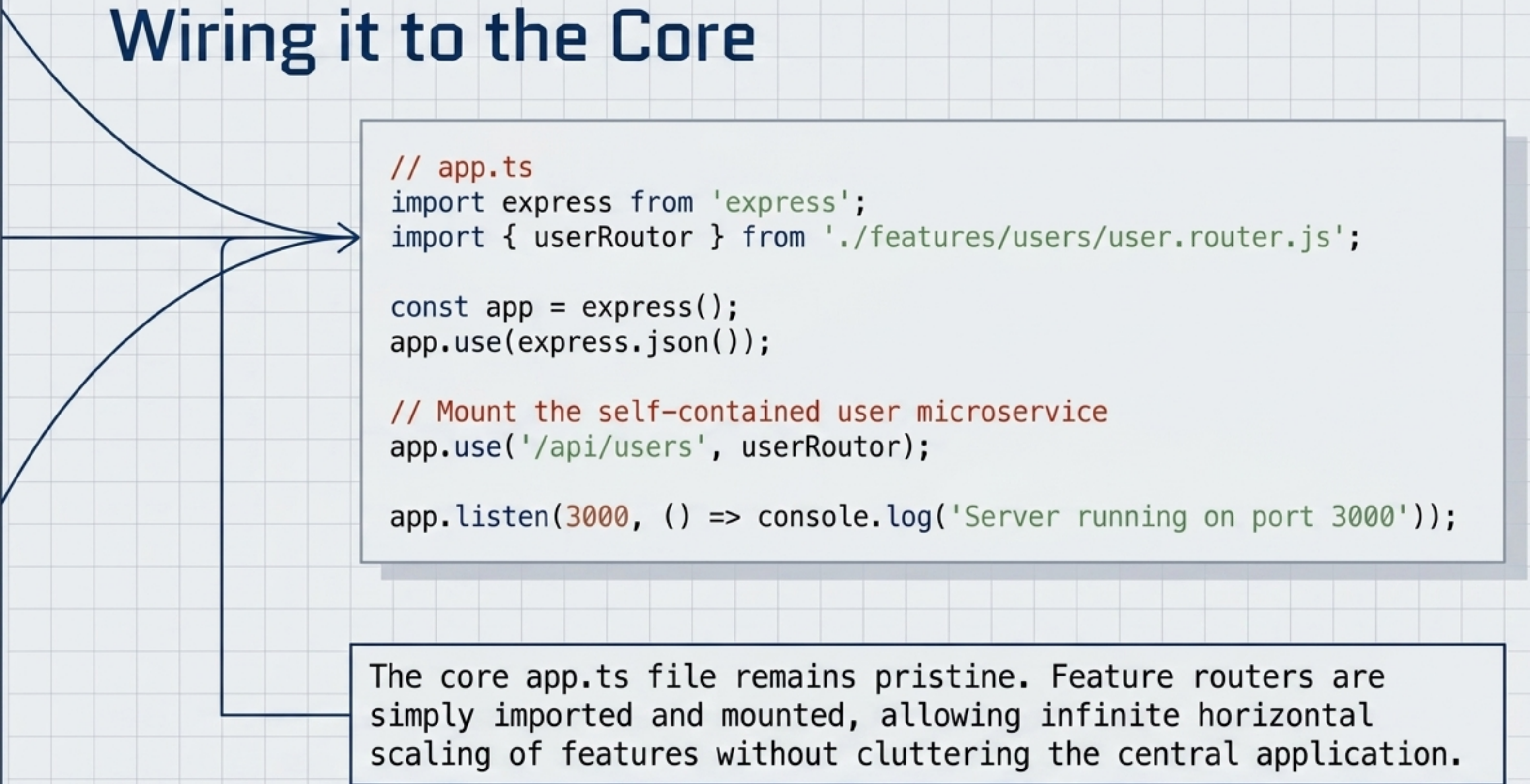
The Request Lifecycle Engine

Isolated Users Domain



Seamless handoffs powered by ES6 imports. The request is processed entirely within the boundaries of the domain, utilizing shared utils only at the final response boundary.

Wiring it to the Core



```
// app.ts
import express from 'express';
import { userRouter } from '../features/users/user.router.js';

const app = express();
app.use(express.json());

// Mount the self-contained user microservice
app.use('/api/users', userRouter);

app.listen(3000, () => console.log('Server running on port 3000'));
```

The core app.ts file remains pristine. Feature routers are simply imported and mounted, allowing infinite horizontal scaling of features without cluttering the central application.

Security & Scale Best Practices



Zero Plaintext Secrets

Never commit .env files. Utilize environment variables safely to prevent supply chain attacks and maintain configuration isolation.



Minimize Dependency Tree

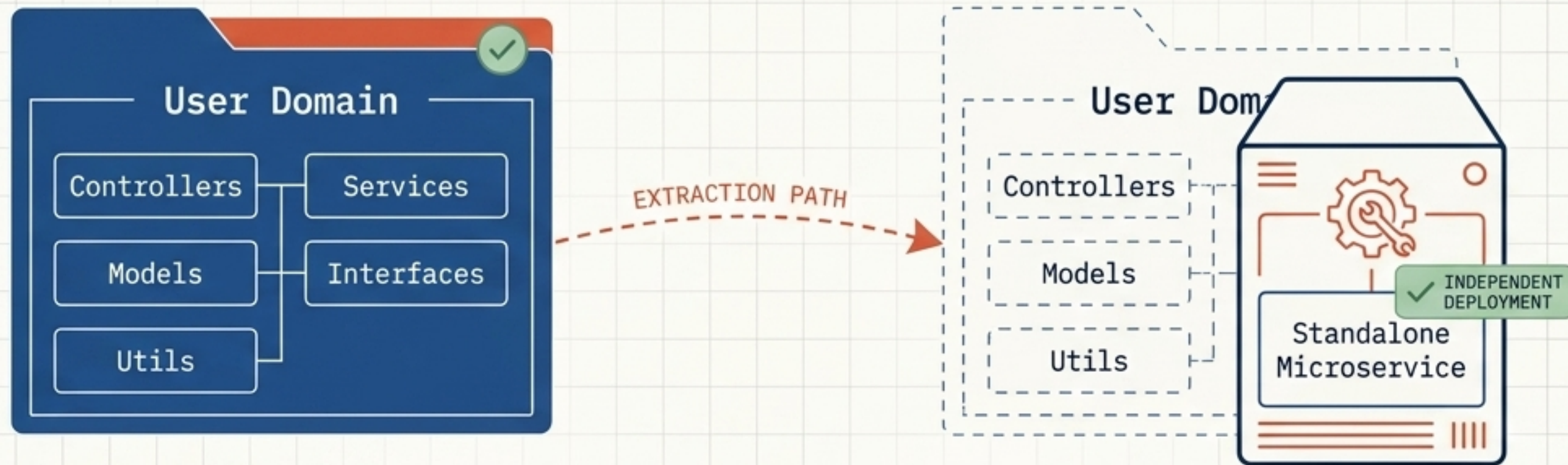
Every NPM package increases your attack surface. Rely on native Node.js and modern ES6 features where possible to reduce bloat.



Automated Updates

Utilize tools like npm-check-updates and Dependabot to proactively secure and patch the simulated microservices environment.

Built for Today. Ready for Tomorrow.



Isolation by Design

The vertical feature slice keeps domain logic entirely decoupled.



Type Safety Everywhere

TypeScript contracts ensure zero data bleed across API boundaries.



The Ultimate Advantage

When the app outgrows the monolith, the domain folder can be effortlessly extracted into a true, standalone **microservice**.